PYTHON 클래스

◆ 데이터 표현 방법

- 학생 데이터

학생 관련 변수들

```
char *name = "홍길동";
char *schoolName = "중학교";
int    schoolNo = 3;
int    banNo    = 1;
int    number   = 10;
```

학생 관련 변수들 하나로 구조체 (structure)

```
struct student
{
    char *name;
    char *schoolName;
    int    schoolNo;
    int    banNo;
    int    number;
}
struct student std = {"홍길동", "중학교", 3, 1, 10}
```

PYTHON 클래스

◆ 데이터 표현 방법

- 학생 데이터

학생 관련 변수 + 함수들 하나로 **클래스(Class)**

```
class Student
{
        char *name;
        char *schoolName;
        int     schoolNo;
        int     banNo;
        int     number;

        char* getName( ) return this.name
        int     getBanNo( ) return this.banNo
}
```

속성/특징

역할/기능

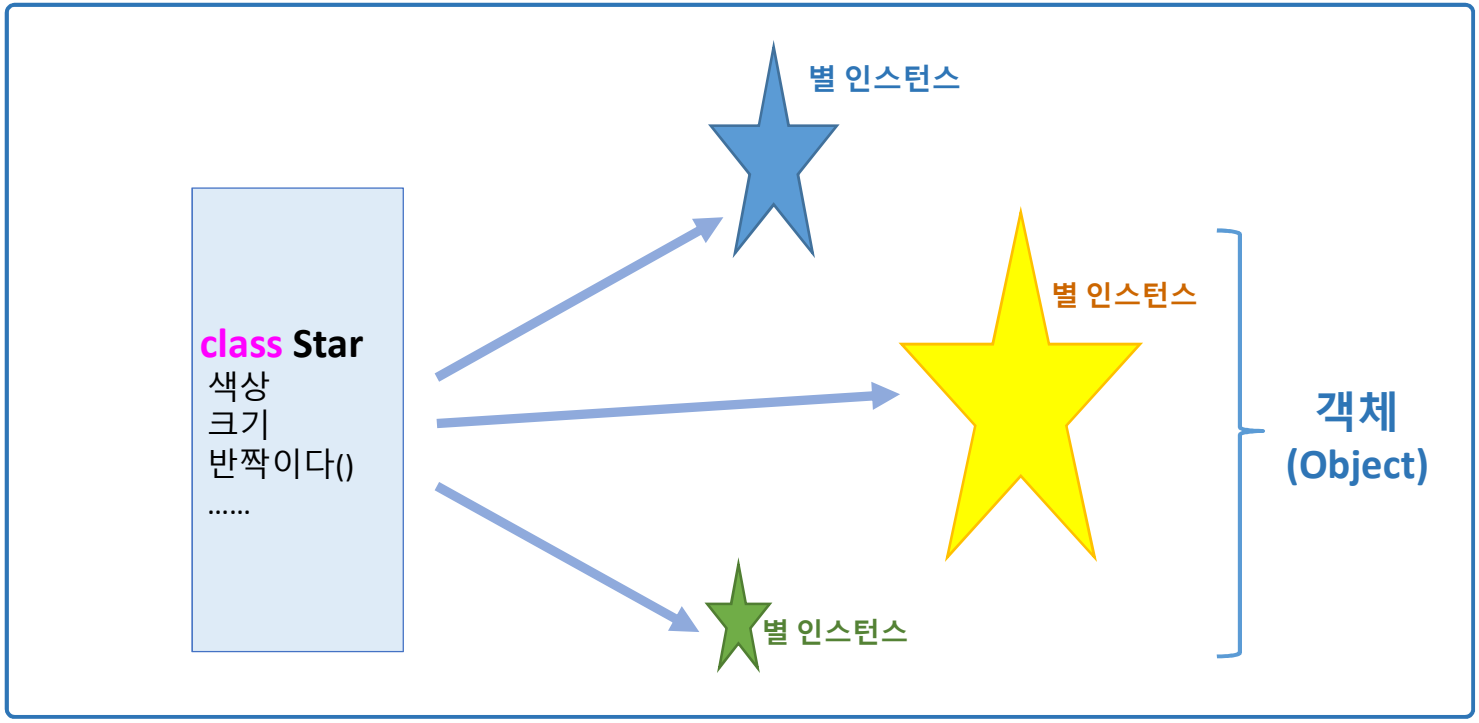
PYTHON 클래스

◆ 클래스(CLASS)

- **특정 데이터를 표현하는 변수와 함수를 하나로 묶어 둔 것**
- 제품의 설계도 또는 틀에 비유
- 설계도에 근거해서 만들어진 제품이 바로 객체
- 제품이 생성된 상세 클래스 정보를 인스턴스

PYTHON 클래스

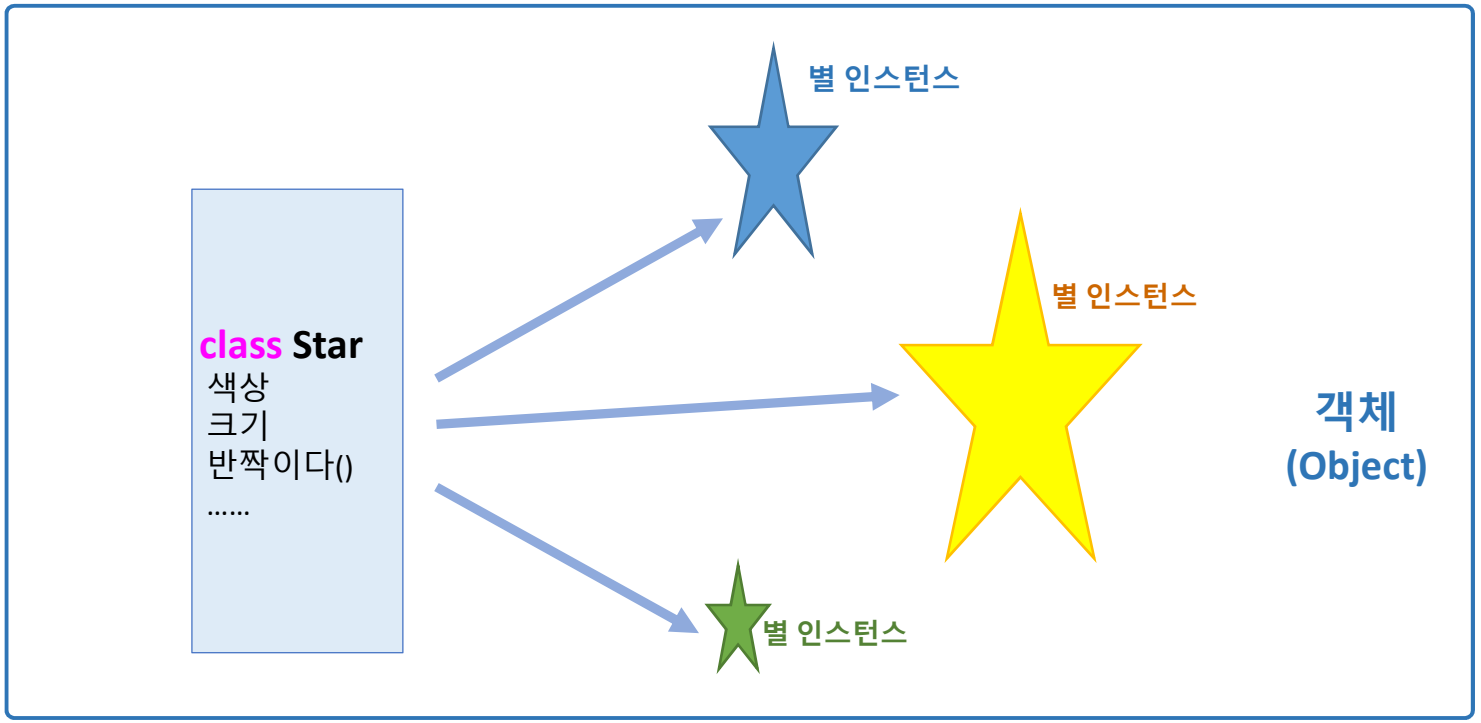
◆ 클래스(CLASS)



PYTHON 클래스

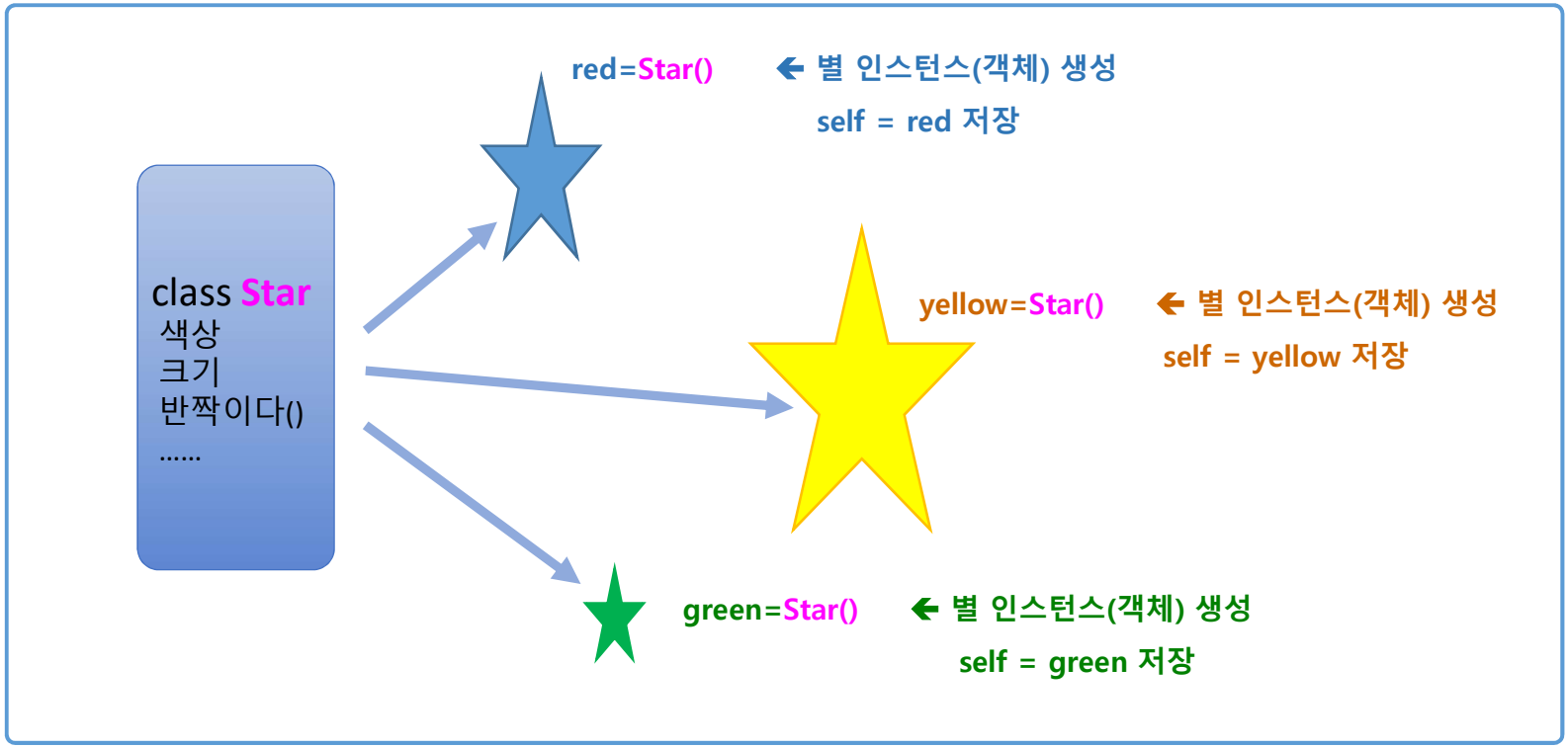
◆ 클래스(CLASS)

- 클래스로부터 생성된 객체 ➔ 인스턴스(Instance)
- 독립적인 데이터를 가짐



◆ 클래스(CLASS)

- **self** 키워드 : **인스턴스 저장된 변수**



PYTHON 클래스

◆ 클래스(CLASS) 구성

class 클래스이름:

클래스 변수 1
:
클래스 변수 N

클래스 속성, 특징, 성질
객체에 **공유되는 변수**

인스턴스 **생성** (메모리 할당)메서드(Constructor)

```
def _ _new_ _(cls, *args, **kwargs):
    instance = super().__new__(cls)
    return instance
```

인스턴스 생성 시 가장 먼저 호출
인스턴스 반환

인스턴스 초기화 메서드

```
def _ _init_ _( self [, 인수1, 인수2,...] ):
```

인스턴스 생성 시 초기화 값
인스턴스 마다 각자의 값 저장

클래스 함수/메서드

```
def 함수명( self [, 인수1, 인수2,...] ):
```

기능, 동작, 행동

◆ 클래스(CLASS) 구성

- 다양한 클래스 생성

```
class ProductNone:
    pass
```

속성도 메서드도 없는 **클래스**

PYTHON 클래스

◆ 클래스(CLASS) 구성

- 다양한 클래스 생성

```
class Product:


    def displayInfo(self, name, price):
        print(f"PNAME = {name}")
        print(f"PRICE = {price}₩n")
```

메서드 만 존재하는 **클래스**

◆ 클래스(CLASS) 구성

```
class Product:

    def __new__(cls, *args, **kwargs):
        print("__new__ called")
        instance = super().__new__(cls)
        print(f'instance => {id(instance)}')
        return instance
```

필드(속성) 생성

인스턴스 생성
메모리 확보

```
    def __init__(self, pname, price):
        self.pname=pname
        self.price=price
```

필드(속성) 생성

인스턴스 변수
초기화

```
    def displayInfo(self, name, price):
        print(f"PNAME = {self.pname}")
        print(f"PRICE = {self.price}₩n")
```

◆ 클래스(CLASS) 구성

- 객체(인스턴스) 생성

객체변수명 = 클래스명()

```
class ProductNone:
    pass
```

생성

```
p1 = ProductNone()
p2 = ProductNone()
```

◆ 클래스(CLASS) 구성

- 객체(인스턴스) 속성 & 메서드 사용

속성값 변경 : 객체변수명.**속성명 = 값**

속성값 읽기 : 객체변수명.**속성명**

메서드 호출 : 객체변수명.**메서드명( )**

◆ 클래스(CLASS) 구성

- 객체(인스턴스) 속성 & 메서드 사용

```
# ---------------------------------------------
# 객체(인스턴스) 생성
# 방법(규칙) :  변수명 = 클래스명()
# ---------------------------------------------
p1=Product('Cake', 'Home', 10000)
p2=Product('Bag', 'PARK', 5000)


# ---------------------------------------------
# 객체(인스턴스)의 메서드 또는 속성(필드) 사용
# 객체(인스턴스)변수명.메서드()
# ---------------------------------------------
p1.displayInfo()                    # 객체( Product 인스턴스 ) 메서드 사용
```

◆ 클래스(CLASS) 구성

- 객체(인스턴스) 속성 & 메서드 사용

```
# ---------------------------------------------
# 객체(인스턴스)의 메서드 또는 속성(필드) 사용
# 객체(인스턴스)변수명.속성명
# ---------------------------------------------
print(f"p1.pmaker =>{p1.pmaker}")      # 객체( Product 인스턴스 ) 필드 값 사용
print(f"p1.price =>{p1.price}")        # 객체( Product 인스턴스 ) 필드 값 사용

p1.price=25000                         # 객체( Product 인스턴스 ) 필드 값 변경
p1.pmaker='Pari'                       # 객체( Product 인스턴스 ) 필드 값 변경

print(f"p1.pmaker =>{p1.pmaker}")      # 객체( Product 인스턴스 ) 필드 값 사용
print(f"p1.price =>{p1.price}")        # 객체( Product 인스턴스 ) 필드 값 사용
```

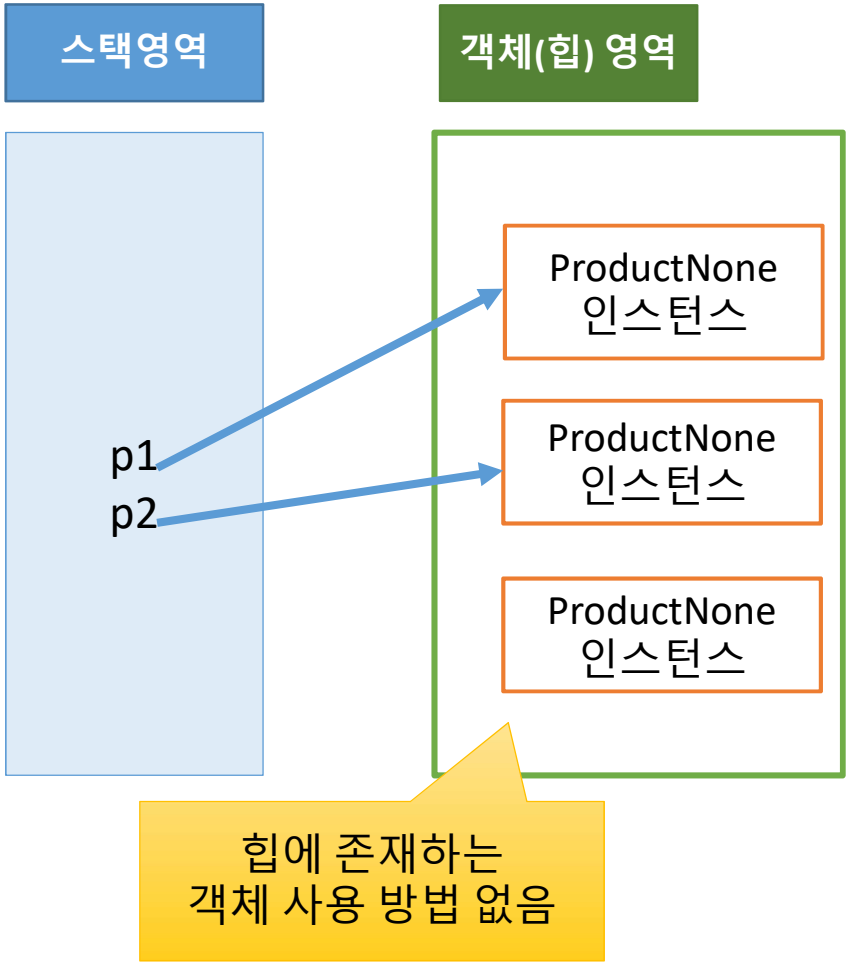
PYTHON 클래스

◆ 객체(인스턴스) 생성

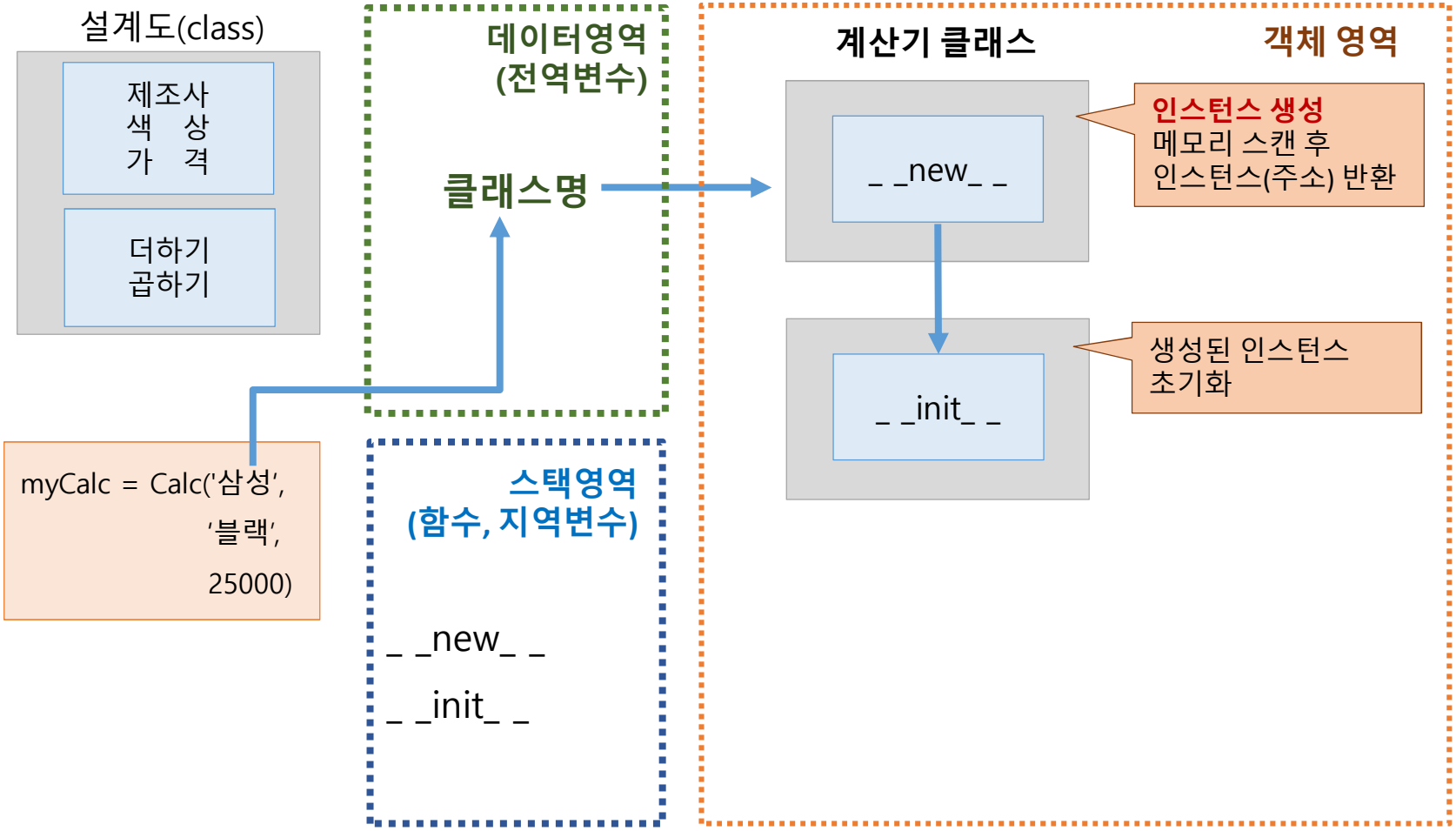
```
class ProductNone:
    pass
```

생성

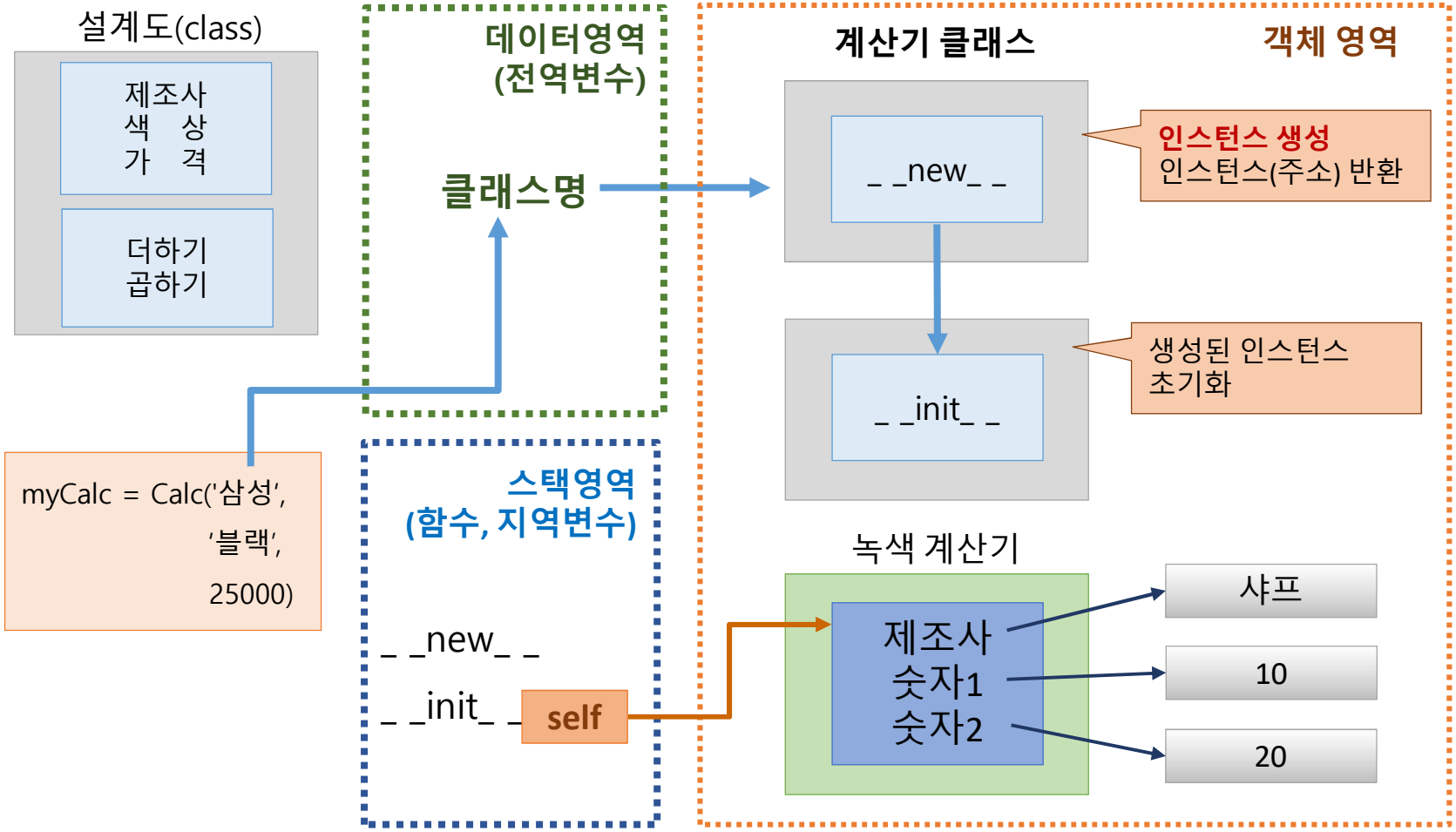
```
p1 = ProductNone()
p2 = ProductNone()
ProductNone()
```



◆ 객체(인스턴스) 생성


설계도(class)
제조사
색 상
가 격
더하기
곱하기
데이터영역
(전역변수)
클래스명
myCalc = Calc('삼성',
'블랙',
25000)
스택영역
(함수, 지역변수)
_ _new_ _
_ _init_ _
계산기 클래스
객체 영역
_ _new_ _
인스턴스 생성
메모리 스캔 후
인스턴스(주소) 반환
_ _init_ _
생성된 인스턴스
초기화

◆ 객체(인스턴스) 생성


설계도(class)
제조사
색 상
가 격
더하기
곱하기
데이터영역
(전역변수)
클래스명
myCalc = Calc('삼성',
'블랙',
25000)
스택영역
(함수, 지역변수)
_ _new_ _
_ _init_ _
self
계산기 클래스
객체 영역
_ _new_ _
인스턴스 생성
인스턴스(주소) 반환
_ _init_ _
생성된 인스턴스
초기화
녹색 계산기
제조사
숫자1
숫자2
샤프
10
20

◆ 클래스(CLASS) 속성과 메서드

- 구성 확인

인스턴스명.__dict__	클래스명.__dict__

```
print(f'p1.__dict__ : {p1.__dict__}')
print(f'Product.__dict__ : {Product.__dict__}')
```

```
p1.__dict__ : {'pname': 'Cake', 'pmaker': 'Home', 'price': 10000}

Product.__dict__ : {'__module__': '__main__', '__init__': <function
Product.__init__ at 0x000001F3C58B2DC8>, 'displayInfo': <function
Product.displayInfo at 0x000001F3C58B2D38>, '__dict__': <attribute
'__dict__' of 'Product' objects>, '__weakref__': <attribute '__weakref__' of
'Product' objects>, '__doc__': None}
```

PYTHON 클래스

◆ 클래스(CLASS) 주요 메서드

❖ 생성자(Constructor)

- 객체(인스턴스) 생성 시 호출
- 인스턴스별 데이터를 메모리에 저장 즉, 초기화
- **def _ _new_ _(cls)**
- **def _ _init_ _(self)**

❖ 소멸자(Destructor)

- 객체(인스턴스) 삭제 시 호출
- 메모리에서 삭제
- **def _ _del_ _(self)**

◆ 클래스(CLASS) 주요 메서드

- 스페셜 메서드 / 매직 메서드

파이썬 시스템에서 **자동 호출되는 메서드**

형태 : **__메서드명__()**

PYTHON 클래스

◆ 클래스(CLASS)와 인스턴스(Instance)

```
class CC:
    name="계산기"

    def add(self, first, second):
        return first + second

    def sub(self, first, second):
        return first - second
```

```
class CCC:
    name = "계산기"

    def __init__(self, first, second):
        self.first=first
        self.second=second

    def add(self):
        return self.first + self.second

    def sub(self):
        return self.first - self.second
```

```
c=CC()
print( "{0} + {1} = {2}".format(10, 20, c.add(10, 20)))
print( "{0} - {1} = {2}".format(10, 20, c.sub(10, 20)))

ccc=CCC(1000, 3000)
print( "{0} + {1} = {2}".format(ccc.first, ccc.second, ccc.add()))
print( "{0} - {1} = {2}".format(ccc.first, ccc.second, ccc.sub()))
```

◆ 클래스(CLASS)와 인스턴스(Instance)

- 변수 종류

인스턴스 변수

- **인스턴스 마다 존재하는 변수**
- 객체변수명으로 읽기 & 변경

비공개 변수 : _ _변수명

- 객체변수명으로 보이지 않음
- **클래스 내에서만 사용 가능**

```
class CCC:

    def __init__(self, datas, age):
        self.data=datas
        self._ _age=age

    def get_total(self):
        total=0
        for i in range(0,len(self.data)):
            total += self.data[i]
        return total
```

◆ 클래스(CLASS)와 인스턴스(Instance)

- 변수 종류

클래스 변수

- **인스턴스들이 공유하는 변수**
- 접근 : **클래스명.변수명**

비공개 변수 : _ _변수명

- 클래스명으로 보이지 않음
- 클래스 내부에서만 사용 가능

```
class CCC:
    _ _share = 1000

    def __init__(self, datas, age):
        self.data=datas
        self._ _age=age

    def get_total(self):
        total=0
        for i in range(0,len(self.data)):
            total += self.data[i]
        return total
```

◆ 클래스(CLASS)와 인스턴스(Instance)

▪ 인스턴스 메서드

- **인스턴스 변수로 접근 가능한 메서드**
- 인스턴스 변수, 클래스 변수 사용 가능한 메서드
- **첫 번째 매개변수: self**

```
def printInfo( self ):
    print(f'반지름 : {self.radius}')
    print(f'원색상 : {self.color}')
    print(f'원넓이 : {Circle.pi * (self.radius**2)}')
    print(f'원넓이 : {Circle.area_formula(self.radius)}')
```

◆ 클래스(CLASS)와 인스턴스(Instance)

▪ 클래스 메서드

- **클래스이름 접근 가능한 메서드**
- 클래스 변수 사용 가능. **클래스 정보 기반 기능 구현 메서드**
- **첫 번째 매개변수: cls, 상단에 데코레이터 @classmethod**

```
@classmethod
def  from_diameter( cls, diameter, color='black'):
        print(f'from_diameter calling')
        radius = diameter / 2
        return cls(radius, color)
```

◆ 클래스(CLASS)와 인스턴스(Instance)

▪ 정적 메서드

- **클래스이름 접근 가능한 메서드**
- 클래스 변수 사용 가능. **인스턴스 & 클래스 정보 필요 없는 기능**
- **상단에 데코레이터 @staticmethod**

```
@staticmethod
def area_formula(radius):
    print(f'area_formula calling')
    print(f'PI => {Circle.pi}')
    return 3.14159 * radius * radius
```

◆ 오버로딩(Overloading)

- 함수 이름 동일
- 매개변수 개수, 타입, 순서가 다른 함수 정의

```
def add(a, b):
    print(a, b)

def add(a, b, c):
    print(a, b, c)

## => 함수명 동일하면 뒤에 정의한 함수 적용
add(10, 20)     # ERROR

## => 이름, 매개변수 갯수 동일, 타입 다름 ==> 오버로딩
add(10, 20, 30)
add('a', 'b', 'c')
```

◆ 연산자 오버라이딩(Overriding)

- **객체**에서 **연산자를 클래스 목적에 맞게 기능 부여** 사용
- 함수이름 앞뒤에 **언더스코어(_ _) 두개** 연속으로 붙은 함수

형태 : **def _ _함수이름_ _( ):**

매직함수명	연산자
def _ _add_ _(self, other)	+
def _ _sub_ _(self, other)	-
def _ _mul_ _(self, other)	*
def _ _truediv_ _(self, other)	/
def _ _floordiv_ _(self, other)	//
def _ _mod_ _(self, other)	%
def _ _pow_ _(self, other)	**

◆ 연산자 오버라이딩(Overriding)

```
# 클래스 생성 ---------------------------
class A:
    def _ _init_ _(self,num):
        self.num=num

# 인스턴스 생성 ---------------------------
a=A(10)
b=A(20)

# + 연산 및 출력 ---------------------------
print(a+b)
-----------------------------------------------------------------
Traceback (most recent call last):
  File "C:/PyChamProject/DAY03/src/EX_Class.py", line 103, in <module>
    print(a+b)
TypeError: unsupported operand type(s) for +: 'A' and 'A'
```

◆ 연산자 오버라이딩(Overriding)

```
# 클래스 생성 ------------------------------
class A:
    def _ _init_ _(self,num):
        self.num=num

    def _ _add_ _(self, other):
        return self.num+other.num

# 인스턴스 생성 ------------------------------
a=A(10)
b=A(20)

# + 연산 및 출력 ------------------------------
print(a+b)
```

◆ Getter & Setter 메서드

- **객체지향언어 특징 중 정보은닉/캡슐화**
  - 클래스 내부의 속성 및 메서드 외부 접근 불허 [기본값]
  - 일부 메서드들만 외부 접근 허용
  - **파이썬 모두 공개[기본값] ➔ _ _ 변수명 , _ _메서드명( )**

◆ Getter & Setter 메서드

- **비공개 변수 접근용 메서드들 ➔ getter & setter method**
- 형식
  - 변수값 설정 : set_변수명(새로운 값)
  - 변수값 읽기 : get_변수명( )

- **반드시 getter/setter 메서드를 추가해야 하는 것은 아님**

◆ Getter & Setter 메서드

```
class Student:
    def __init__(self, name):
        self._ _name = name     # 비공개 속성

    def get_name(self):         # 속성 간접 접근
        return self._ _name

    def set_name(self, value):  # 속성 간접 접근
        self._ _name = value
```

◆ Getter & Setter 메서드

```
## 인스턴스 생성
std1=Student1("홍길동")

## 공개 속성 직접 접근
std1.공개속성명 = '새로운값'
print ( std1.공개속성명 )

## 비공개 속성 직접 접근 불가
#print(std1.__name)
## => 간접 접근 getter/setter method
print( std1.get_name() )
std1.set_name("홍이장군")
```

◆ Getter & Setter 메서드

- **데코레이터 적용**
- **형식**
  - 변수값 읽기 : @property , 변수명을 함수명으로
  - 변수값 설정 : @변수명.setter , 변수명을 함수명으로

◆ Getter & Setter 메서드

```
class Student:
    def __init__(self, name):
        self._ _name = name    # 속성 직접 노출하지 않음

    @property
    def name(self):
        return self._ _name      # getter 역할

    @name.setter
    def name(self, value):
        self._ _name = value     # setter 역할
```

◆ Getter & Setter 메서드

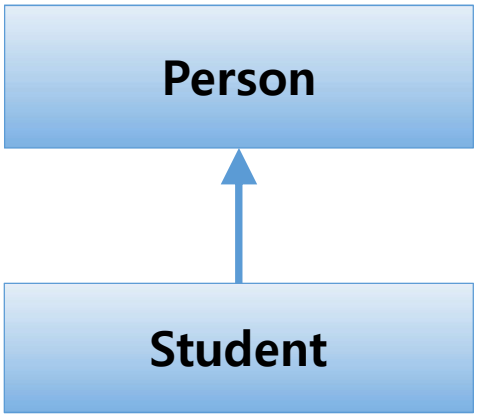
```
## 인스턴스 생성
std2=Student2("마징가")

##=> 간접 접근 @property
std2.name = "MAMAMu"
print(f'std2 name : {std2.name}')
```

◆ 상속(inheritance)

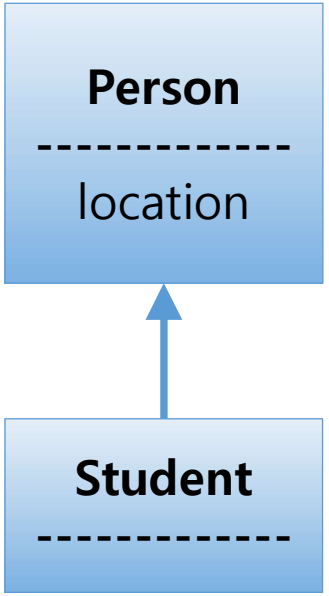
```
class Person:
        pass

class Student(Person) :
         pass
```




```
# 상속 관계 확인하기 ----------------------------------------------------------
print(f"Student는 Person의 자식? { issubclass(Student, Person)}")
```

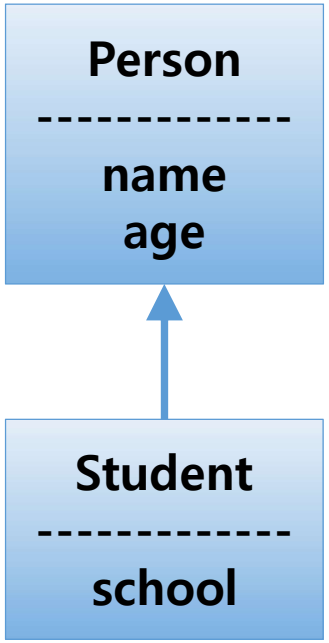
◆ 상속(inheritance)




```
class Person:
    def __init__(self):
        self.location='Korea'

class Student(Person):
    def showInfo(self):
        print(f"Location : {self.location}")
```

◆ 상속(inheritance)




```
class Person:
    def __init__(self, name, age):
        print("Person __init__() ")
        self.name=name
        self.age=age

class Student(Person):
    def __init__(self, name, age, school):
        super().__init__(name, age)
        print("Student __init__() ")
        self.school=school

def showInfo(self):
        print(f"name : { self.name}₩tschool : {self.school}")
```

◆ 상속(inheritance)

- **메소드 오버라이딩(Overring)**

함수 구현 부분만 다시 **재정의**

상속관계에서 **부모에서 상속 받은 메소드**에 한정

◆ 상속(inheritance)

```
class Point:
    def __init__(self, x, y):
        self.x=x
        self.y=y

    def show_point(self):
        return f'({self.x}, {self.y})'
```

```
class DPoint(Point):
    def __init__(self, x, y, z):
        super().__init__(x, y)
        self.z = z

    def show_point(self):
        return f'({self.x}, {self.y}, {self.z})'
```

오버라이딩

PYTHON 클래스

◆ Object 클래스

- 모든 클래스의 부모 클래스
- 자동으로 상속받게 되는 클래스

```
__class__                                  object
__str__(self)                              object
__annotations__                            object
__delattr__(self, name)                    object
__dir__(self)                              object
__eq__(self, o)                            object
__format__(self, format_spec)              object
__getattribute__(self, name)               object
__hash__(self)                             object
__init_subclass__(cls)                     object
__ne__(self, o)                            object
__new__(cls)                               object
```